4 More Redirection

As we learned in Chapter 6 of TLCL, I/O redirection is one of the most useful and powerful features of the shell. With redirection, our commands can send and receive streams of data to and from files and devices, as well as allow us to connect different programs together into pipelines.

In this adventure, we will look at redirection in a little more depth to see how it works and to discover some additional features and useful redirection techniques.

What's Really Going On

Whenever a new program is run on the system, the kernel creates a table of *file descriptors* for the program to use. File descriptors are pointers to files. By convention, the first 3 entries in the table (descriptors 0, 1, and 2) are used as standard input (stdin), standard output (stdout), and standard error (stderr). Initially, all three descriptors point to the terminal device (which the system treats as a read/write file), so that standard input comes from the keyboard and standard output and standard error go to the terminal display.

When a program is started as a child process of another (for instance, when we run an executable program in the shell), the newly launched program inherits a copy of the parent's file descriptor table. Redirection is the process of manipulating the file descriptors so that input and output can be routed from/to different files.

The shell hides the presence of file descriptors in common redirections such as:

command > *file*

Here we redirect standard output to a file, but the full syntax of the redirection operator includes an optional file descriptor. We could write the above statement this way and it would have exactly the same effect:

command 1> *file*

As a convenience, the shell assumes we want to redirect standard output if the file descriptor is omitted. Likewise, the following two statements are equivalent when referring to standard input:

command < *file*

command 0< *file*

Duplicating File Descriptors

It is sometimes desirable to write more than one output stream (for example standard output and standard error) to the same file. To do this, we would write something like this:

command > *file* 2>&1

We'll add the assumed file descriptor to the first redirection to make things a little clearer:

command 1> *file* 2>&1

This is an example of *duplication*. When we read this statement, we see that file descriptor 1 is changed from pointing to the terminal device to instead pointing to *file*. This is followed by the second redirection that causes file descriptor 2 to be a duplicate (i.e., it points to the same file) of file descriptor 1. When we look at things this way, it's easy to see why the order of redirections is important. For example, if we reverse the order:

command 2>&1 1> *file*

file descriptor 2 becomes a duplicate of file descriptor 1 (which points to the terminal) and then file descriptor 1 is set to point to *file*. The final result is file descriptor 1 points to *file* while file descriptor 2 still points to the terminal.

exec

Before we go any farther, we need to take a brief detour and talk about a shell builtin that we didn't cover in TLCL. This builtin is named `exec` and it does some interesting things. Its main purpose is to terminate the shell and launch another program in its place. This is often used in startup scripts that initiate system services. However, it is not common in scripts used for other purposes.

Usage of `exec` is described below:

`exec` [*program*] [*redirections*]

program is the name of the program that will start and take the place of the shell. *redirections* are the redirections to be used by the new program.

One feature of `exec` is useful for our study of redirection. If *program* is omitted, any specified redirections are performed on the current shell. For example, if we included this near the beginning of a script:

```
exec 1> output.txt
```

from that point on, every command using standard output would send its data to `output.txt`. It should be noted that if this trick is performed by a script, it is no longer

possible to redirect that script's output at runtime using the command line. For example, if we had the following script:

```
#!/bin/bash

# exec-test - Test external redirection and exec

exec 1> ~/foo1.txt
echo "Boo."

# End of script
```

and tried to invoke it with redirection:

```
me@linuxbox ~ $ ./exec-test > ~/foo2.txt
```

the attempted redirection would have no effect. The word "Boo" would still be written to the file `foo1.txt`, not `foo2.txt` as specified on the command line. This is because the redirection performed inside the script via `exec` is performed after the redirection on the command line, and thus, takes precedence.

Another way we can use `exec` is to open and close additional file descriptors. While we most often use descriptors 0, 1, and 2, it is possible to use others. Here are examples of opening and closing file descriptor 3:

```
# Open fd 3
exec 3> some_file.txt

# Close fd 3
exec 3>&-
```

It's easy to open and use file descriptors 3-9 in the shell, and it's even possible to use file descriptors 10 and above, though the `bash` man page cautions against it.

So why would we want to use additional file descriptors? That's a little hard to answer. In most cases we don't need to. We *could* open several descriptors in a script and use them to redirect output to different files, but it's just as easy to specify (using shell variables, if desired) the names of the files to which we want to redirect since most commands are going to send their data to standard output anyway.

There is one case in which using an additional file descriptor would be helpful. It's the case of a filter program that accepts standard input and sends its filtered data to standard output. Such programs are quite common, for example `sort` and `grep`. But what if we want to create a filter program that also writes stuff on the terminal display while it was filtering? We can't use standard output to do it, because standard output is being used to output the filtered data. We could use standard error to display stuff on the screen, but let's say we wanted to keep it restricted to just error messages (this is good for logging). Using `exec`, we could do something like this:

```
#!/bin/bash

# counter-exec - Count number of lines in a pipe
```

```
exec 3> /dev/tty # open fd 3 and point to controlling terminal

count=0
while read; do  # read line from stdin
  echo "$REPLY" # send line to stdout
  ((count++))
  printf "\b\b\b\b\b\b%06d" $count >&3
done
echo " Lines Counted" >&3

exec 3>&- # close fd 3
```

This program simply copies standard input to standard output, but it displays a running count of the number of lines that it has copied. If we invoke it this way, we can see it in action:

```
me@linuxbox ~ $ find /usr/share | ./counter-exec > ~/find_list.txt
```

In this pipeline example, we generate a list of files using `find`, and then count them before writing the list in a file named `find_list.txt`.

The script works by reading a line from the standard input and writing the `REPLY` variable (which contains the line of text from `read`) to standard output. The `printf` format specifier contains a series of six backspaces and a formatted integer that is always six digits long padded with leading zeros.

/dev/tty

The mysterious part of the script above is the `exec`. The `exec` is used to open a file using file descriptor 3 which is set to point to `/dev/tty`. `/dev/tty` is one of several *special files* that we can access from the shell. Special files are usually not “real” files in the sense that they are files that exists on a physical disk. Rather, they are virtual like the files in the `/proc` directory. The `/dev/tty` file is a device that always points to a program’s *controlling terminal*, that is, the terminal that is responsible for launching the program. If we run the command `ps aux` on our system, we will see a listing of every process. At the top of the listing is a column labeled “TTY” (short for “Teletype” reflecting its historical roots) that contains the name of the controlling terminal. Most entries in this column will contain “?” meaning that the process has no controlling terminal (the process was not launched interactively), but others will contain a name like “pts/1” which refers to the device `/dev/pts/1`. The term “pty” means *pseudo-terminal*, the type of terminal used by terminal emulators rather than actual physical terminals.

Noclobber

When the shell encounters a command with output redirection, such as:

command > *file*

the first thing that happens is that the output stream is started by either creating *file* or, if *file* already exists, truncating it to zero length. This means that if *command* completely fails or doesn't even exist, *file* will end up with zero length. This can be a safety issue for new users who might overwrite (or truncate) a valuable file.

To avoid this, we can do one of two things. First we can use the ">>" operator instead of ">" so that output will be appended to the end of *file* rather than the beginning. Second, we can set the "noclobber" shell option which prevents redirection from overwriting an existing file. To activate this, we enter:

```
set -o noclobber
```

Once we set this option, attempts to overwrite an existing file will cause the following error:

```
bash: file: cannot overwrite existing file
```

The effect of the `noclobber` option can be overridden by using the `>|` redirection operator like so:

command `>|` *file*

To turn off the noclobber option we enter this command:

```
set +o noclobber
```

Summing Up

While this adventure may be more on the "interesting" side than the "fun" side, it does provide some useful insight into how redirection actually works and some of the interesting ways we can use it. In a later adventure, we will put this new knowledge to work expanding the power of our scripts.

Further Reading

- A good visual tutorial can be found at The Bash Hackers Wiki: https://wiki.bash-hackers.org/howto/redirection_tutorial
- For a little background on file descriptors, see this Wikipedia article: https://en.wikipedia.org/wiki/File_descriptor
- This *Linux Journal* article covers using `exec` to manage redirection: https://www.linuxjournal.com/content/bash-redirections-using-exec
- *The Linux Command Line* covers redirection in Chapters 6 (main discussion), 25 (here documents), 28 (here strings), and 36 (command grouping, subshells, process substitution, named pipes).
- The REDIRECTION section of the `bash` man page, of course, has all the details.